Теория автоматов и формальных языков

Алгоритмически неразрешимые проблемы

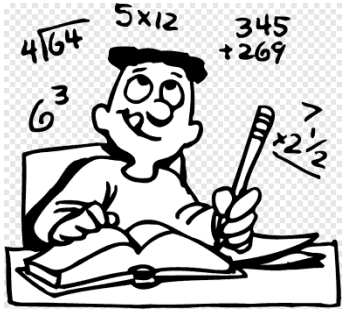
Алгоритм - фундаментальная концепция информатики

Что такое АЛГОРИТМ?

У каждого свое интуитивное понимание того, что это такое.

- **Одно определение**. *Алгоритм - это определенное на некотором языке конечное предписание (способ, рецепт), задающее дискретную последовательность исполнимых элементарных операций над входными данными. Процесс выполнения предписания состоит из отдельных шагов, на каждом из которых выполняется одна очередная операция*
- Но что такое “процедура”, “инструкция”?
- С какими объектами работает алгоритм: числами, матрицами, графами, таблицами?
- **Нужно иметь формальное определение алгоритма**, чтобы точно знать, что является алгоритмом, а что нет.

- Слово ”алгоритм” возникло довольно поздно (оно образовано от имени арабского математика аль-Хорезми, жившего в 9 веке н. э.)



Алгоритм

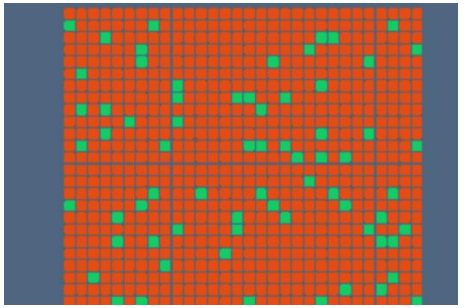
- Алгоритм — это конечный набор четких, недвусмысленных инструкций, следуя которым можно по входным данным определенного вида получать на выходе некоторый результат.

- По сути алгоритмы — это ”продукт” математики, как древней, так и современной, то, что она дает на выходе.

Ситуация:

- Некто приносит математику задачу и просит, чтобы он предоставил ему *алгоритм ее решения*. Математик, подумав, может дать ему этот алгоритм.
- Может случится так, что он через некоторое время скажет: ”Ничего не могу придумать. Задача что-то очень сложная. Буду думать еще.”
- Но возможен и третий вариант: математик, подумав, скажет: ”Я тут провел исследования и доказал, что алгоритма для решения вашей задачи не существует.”

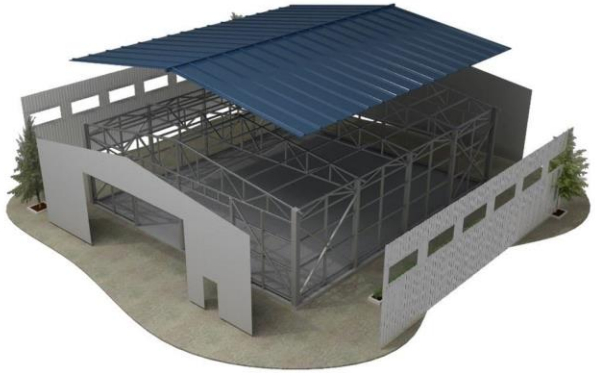
Неразрешимые задачи



- Классические неразрешимые задачи на построение при помощи циркуля и линейки: удвоение куба, трисекция угла, квадратура круга.
- Решение диофантовых уравнений (т. н. 10-ая проблема Гильберта).
  - Доказано, что не существует алгоритма, который по многочлену от нескольких переменных с целыми коэффициентами определял бы, есть ли у него целочисленные корни.
- Неразрешимость элементарной теории арифметики.
  - Не существует алгоритма, который бы по утверждению о натуральных числах выдавал ответ на вопрос: истинно оно или ложно?

Игра «Жизнь»: *Можно ли по начальному расположению живых клеток сказать, сохранится ли жизнь на поле через 1 000 000 итераций?*

Конструктивные объекты



единый подход к определению формата аргументов и результатов алгоритма.

Конструктивными объектами являются символы, логические значения, целые и вещественные числа, представимые в машине, массивы конструктивных объектов.

Алгоритм (его текст) также является *конструктивным объектом* и, значит, может рассматриваться как данные для другого алгоритма: текст программы (алгоритма) является входными данными для программы-транслятора.

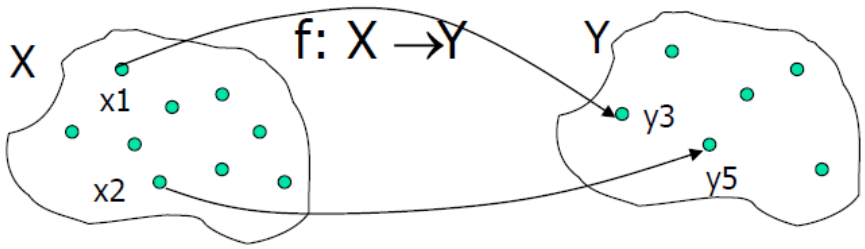
- ***Конструктивным объектом*** называется такой объект, который может быть полностью описан при помощи конечной последовательности символов.
- ***Конструктивным простра***нством называется множество всех конструктивных объектов одинакового вида.
- Вычислимой функцией называется функция из одного конструктивного пространства в другое конструктивное пространство, для которой существует алгоритм, позволяющий по описанию любого аргумента получить описание значения функции на этом аргументе.

Алгоритмы преобразования объектов

Все вопросы, касающиеся алгоритмов преобразования объектов одного конструктивного пространства в объекты другого конструктивного пространства, можно разбить на четыре категории:

- Вопросы, относящиеся собственно к ***теории вычислимости***. Это вопросы о том, какие функции являются вычислимыми, а какие нет, и вопросы: "Какие функции станут вычислимыми, если мы объявим вычислимой какую-нибудь конкретную функцию?"
- Вопросы, относящиеся к ***теории сложности***. Это вопросы о том, как быстро (за какое количество шагов) можно вычислить ту или иную вычислимую функцию, то есть вопросы о сложности оптимальных алгоритмов.
- Вопросы, относящиеся к ***программированию*** или, более широко, к *computer science*. Это вопросы относятся к конкретным алгоритмам вычислений и к их реализации.
- Вопросы, относящиеся к ***методам вычислений***. Это вопросы о нахождении приближенных решений различных дифференциальных и интегральных уравнений при помощи цифровых алгоритмов.

Алгоритм как отображение



Алгоритм – способ реализации отображения
f: X →Y
где X и Y – множества произвольных объектов

Для того чтобы построить единое формальное определение алгоритма, нужно сначала задать способ описания входных аргументов и выходных результатов

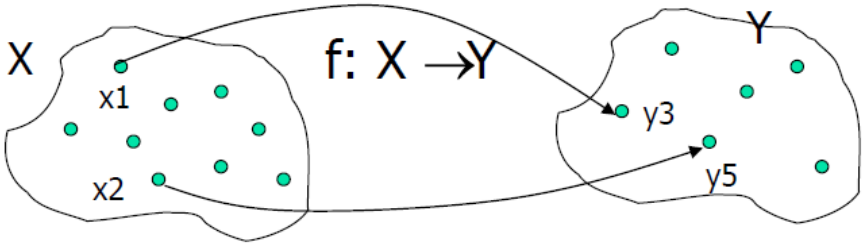
Исходными данными и результатами алгоритма могут быть разные объекты: числа (натуральные, целые, вещественные, ...), вектора, матрицы, графы, строки, формулы, графики, ...

Можно ли придумать единый способ задания аргументов и результатов?

Количество возможных функций N→N бесконечно (континуум)

Конструктивные объекты в алгоритмах

X, Y - вектора, матрицы, графы, …



- КОНСТРУКТИВНЫЕ объекты – простейшие, такие, которые можно сконструировать, построить из конечного числа базовых элементов элементарной процедурой конструирования
- В качестве конструктивных объектов выбираются
  натуральные числа
  цепочки конечного словаря
  - Натуральные числа строятся процедурой s(x) – следующий, из 0
  - Цепочки: обычно конечный словарь (алфавит) $\Sigma = \{a, b, c\}$, Базовые объекты – сисмволы, процедура конструирования – операция конкатенации '•' – присоединение символа к цепочке: aabac • a = aabaca
  - Множество всех цепочек из символов Σ: $\Sigma^* = \{\varepsilon, a, b, bb, \ldots, bccab, \ldots\}$. Множество Σ^* является бесконечным (счетным)

Цепочками можно задать любые структуры данных

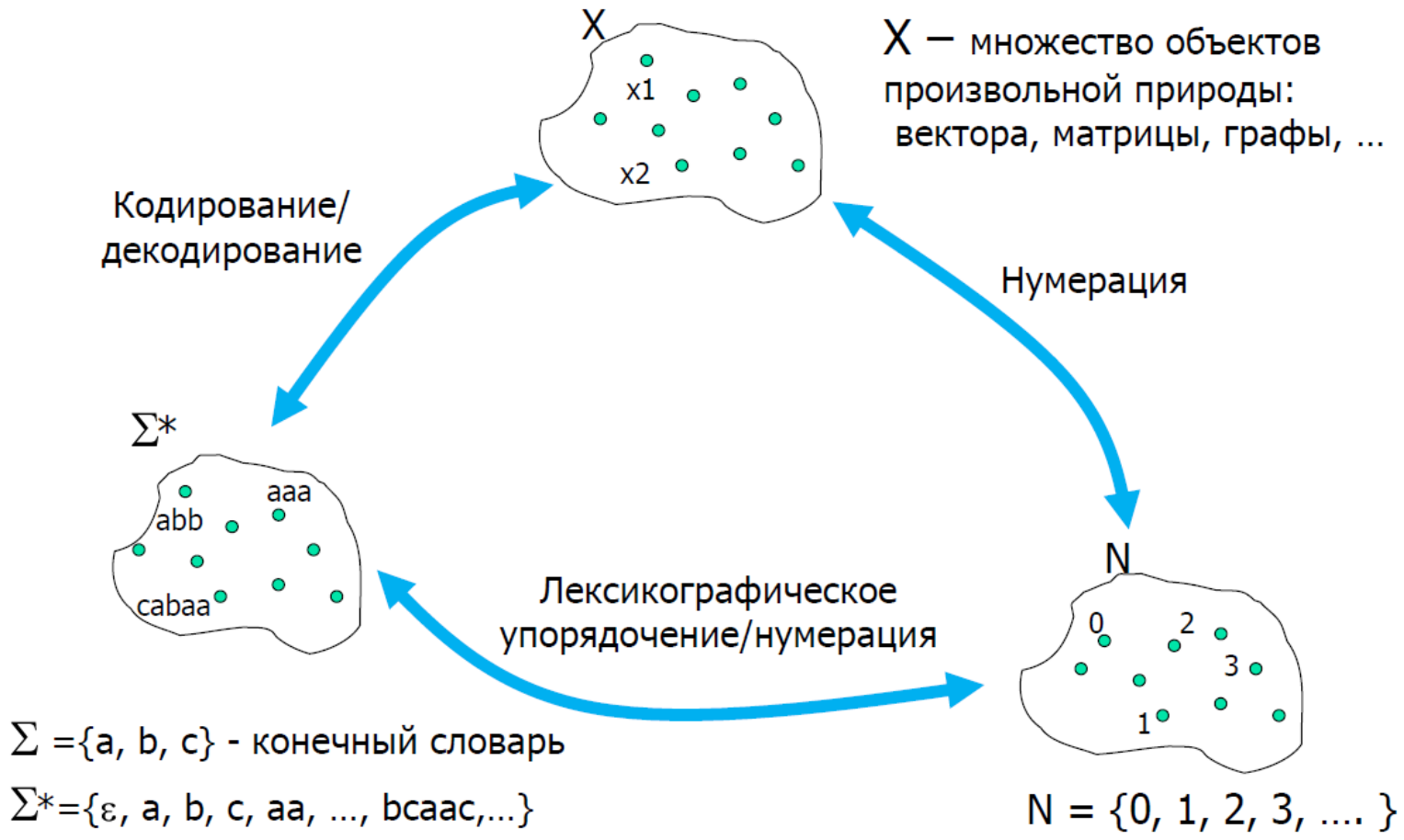
Если мы хотим записать алгоритм, применяемый к парам слов или к цепочкам слов, мы должны записать такие аргументы в виде слов в каком-нибудь алфавите

Например, обозначим символом '*' такой символ, который не принадлежит алфавиту Σ. Тогда цепочку слов <C1, …,Cn> слов в алфавите Σ можно считать словом в алфавите $\Sigma \cup \{'*'\}$

Таким образом, можно считать, что исходные данные и результаты ЛЮБОГО алгоритма просто словами в конечном словаре, т.е. всегда можно задать словарь исходных данных и словарь результатов



Как однотипно задать элементы множества аргументов и множества результатов в алгоритмах?



Кодирование

Пусть M – не более, чем счетное множество объектов любой природы, а Σ - конечный словарь

Кодирование – это однозначное соответствие λ: M→Σ^*.

Примеры кодирования:

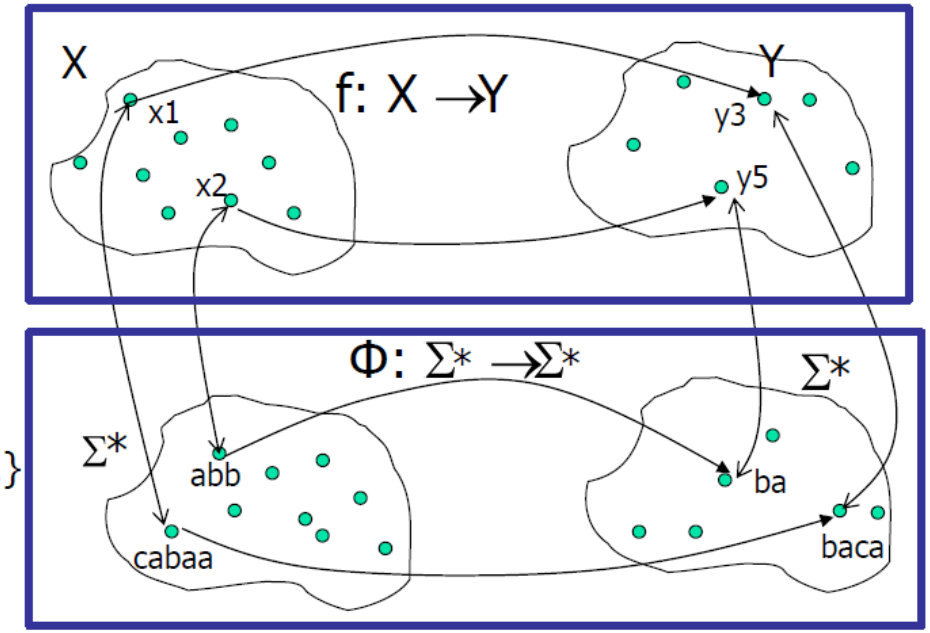
- - унарный код для натуральных чисел:
  Σ = {|}, числу 5 соответствует цепочка '|||||'
- - двоичный код для натуральных чисел:
  Σ = {0,1}, числу 5 соответствует цепочка '101'



Кодирование объектов цепочками

Графы, матрицы, вектора, формулы, графики, … . любые структуры данных можно задать цепочками символов из конечного словаря

X, Y - вектора, матрицы, графы, …



$\Sigma = \{a, b, c\}$

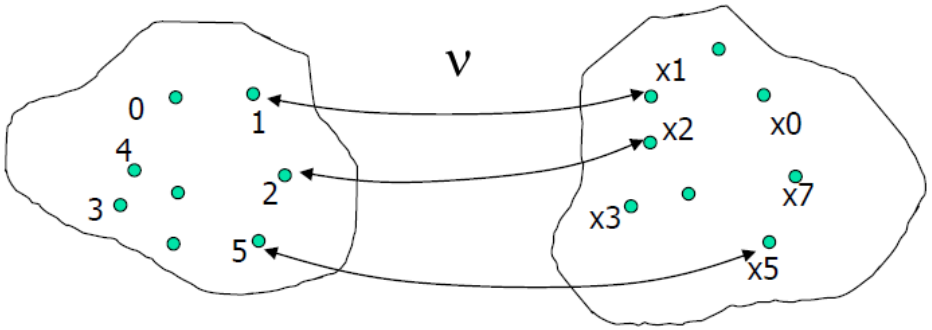
$\Sigma^* = \{\varepsilon, a, b, c, aa, \ldots, bcaac, \ldots\}$

Σ - конечный словарь

Любой алгоритм f: X→Y можно определить, как преобразование над цепочками конечного словаря Φ: $\Sigma^* \to \Sigma^*$

Нумерация множеств натуральными числа

- *Нумерацией множества* X называется всякое взаимно однозначное отображение $\nu : N \to X$ т.е. сопоставление номеру $n \in N$ элемента $x_k \in X$



- Для множеств элементарных объектов процесс нумерации не представляет сложностей

Любой алгоритм можно определить, как преобразование над натуральными числами

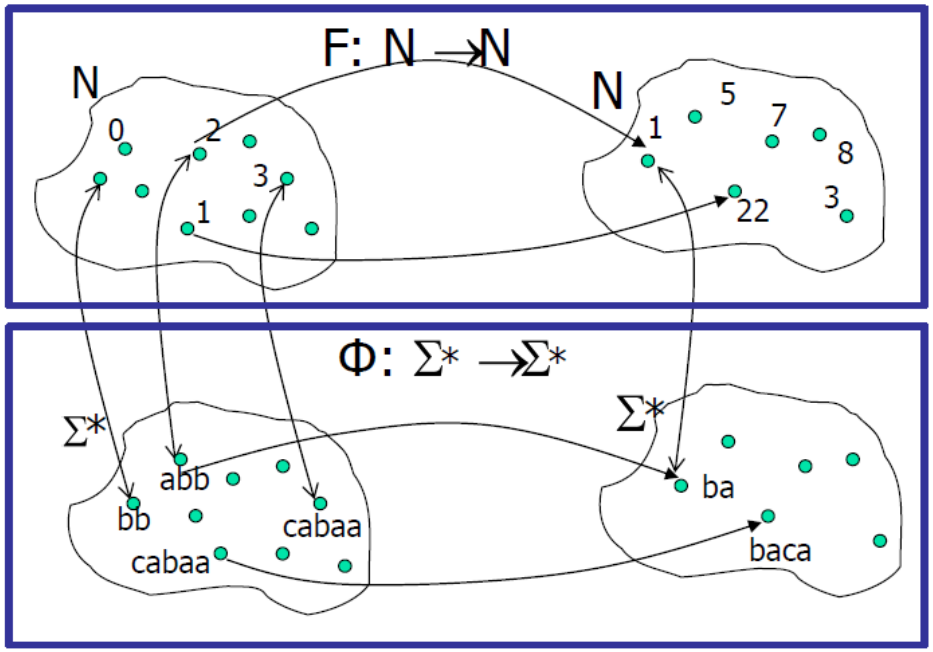
Кодирование натуральных чисел цепочками

С помощью лексикографического упорядочения можно установить взаимно-однозначное соответствие между цепочками конечного словаря и натуральными числами

$N = \{0, 1, 2, 3, \ldots.\}$

$\Sigma = \{a, b, c\}$

$\Sigma^* = \{\varepsilon, a, b, c, aa, \ldots, bcaac, \ldots\}$

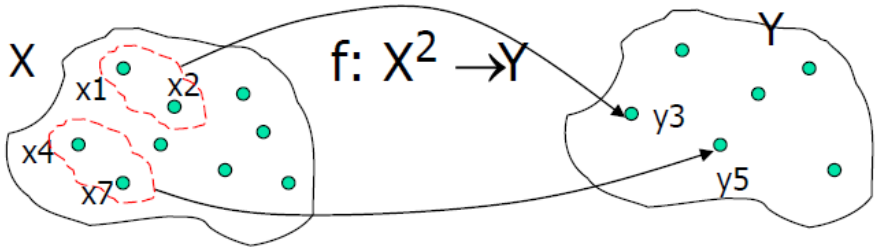


Любую функцию над натуральными числами F: N→N можно определить, как преобразования цепочек Φ: Σ*→Σ*, и наоборот

Нумерация для n-местных функций

Пусть с помощью нумерации установили соотношение между не более, чем счетным множеством X и множеством натуральных чисел N

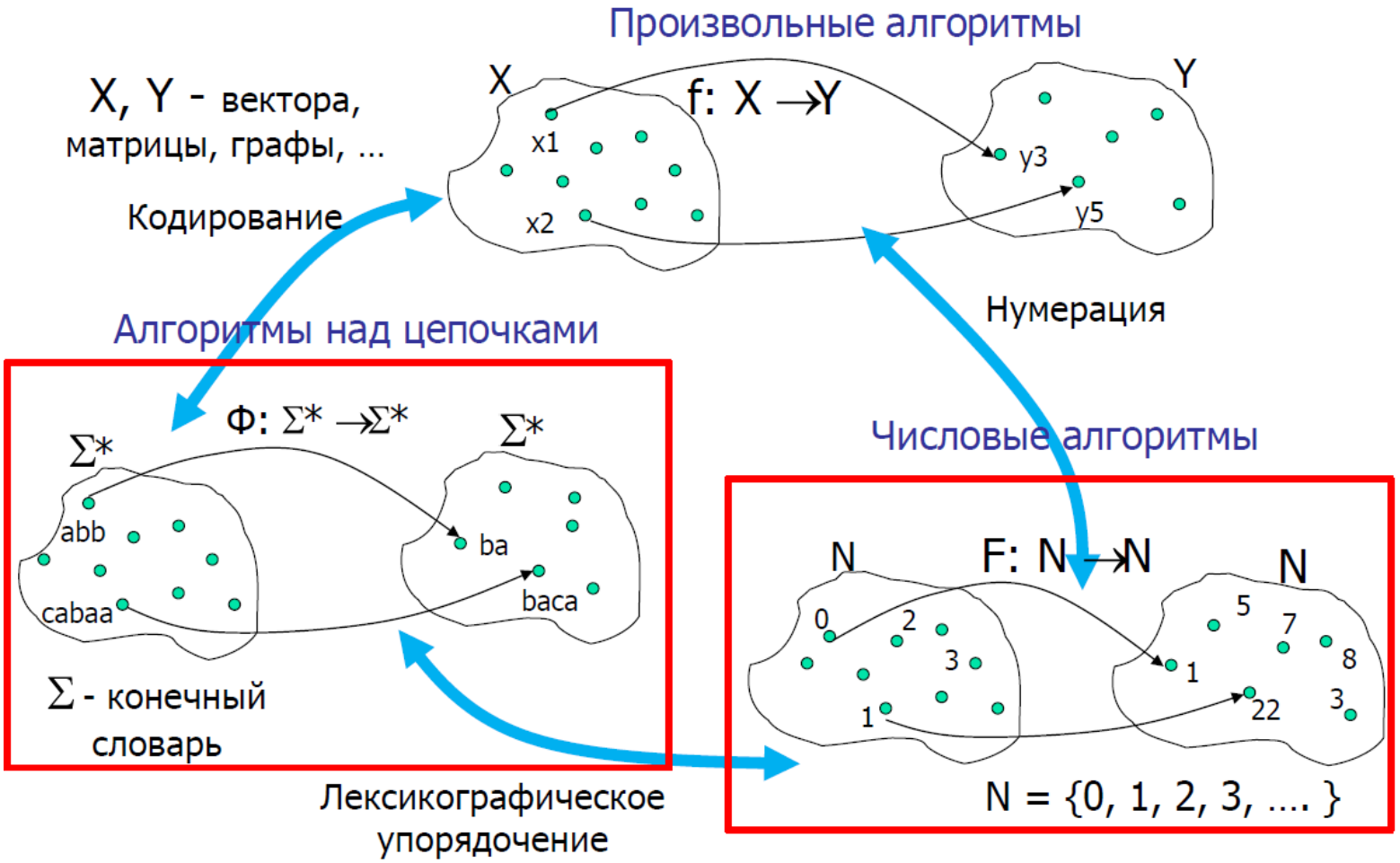
X, Y - вектора, матрицы, графы, …



все определения всех чисел, с которыми мы имеем и когда-либо будем иметь дело в математике, задаются конечными последовательностью слов состоящих из букв конечного алфавита

- Как быть, если в качестве аргументов – пары элементов счетных множеств?
- Например, функция умножения двух матриц f: M×M →M
- Можно ли однозначно пронумеровать пары, тройки,… элементов счетных множеств??

Анализ алгоритмов над произвольными структурами данных



Алгоритм - фундаментальная концепция информатики

- **Определение.** Алгоритм - это дискретная последовательность исполнимых элементарных операций над натуральными числами. Процесс выполнения предписания состоит из отдельных шагов, на каждом из которых выполняется одна очередная операция.
- Но что такое “исполнимая элементарная операция”?
- Выбор набора “исполнимых элементарных операций” для разных определений алгоритма может быть различным.

Три типа универсальных алгоритмических моделей

Первый тип: алгоритмы вида $F: N \to N$

числовые функции на множестве натуральных чисел

Наиболее изученная модель этого типа - рекурсивные функции (Алонзо Черч) - первая формализация понятия алгоритма. Берется набор из трех простейших базовых функций и несколько операции над ними

Второй тип: алгоритмы преобразования цепочек вида $\Phi: \Sigma^* \to \Sigma^*$

пошаговые преобразования слов в произвольных алфавитах

Элементарные операции – подстановки $\alpha \to \beta$, т.е. замены части α входного слова другим словом β.

Примеры этого типа - нормальные алгоритмы Маркова, системы Поста

Третий тип: алгоритмы преобразования цепочек вида $\Phi: \Sigma^* \to \Sigma^*$

представление алгоритма, как автомата над множеством входных цепочек

автомат - устройство, читающее по шагам символы входного слова, и выдающее на выход символы выходного слова в зависимости от его состояния и входного символа.

Примеры этого типа – конечный автомат, машина Тьюринга, машина Поста



Представление алгоритмов рекурсивными арифметическими функциями

Числовые функции, значение которых можно установить посредством некоторого алгоритма, называются вычислимыми.

- А.Черч предложил определить все вычислимые функции как арифметические (целочисленные) функции, которые могут быть конструктивно (с помощью рекурсии) определены через простейшие. Они были названы ***рекурсивными функциями***

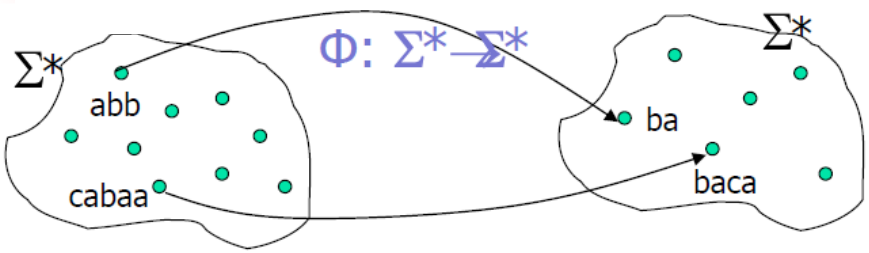


Представление алгоритмов системами преобразования цепочек

Нормальные алгоритмы Маркова



Нормальные алгоритмы Маркова



Самая простая идея: представить алгоритм как конечное число правил простейших преобразований входного слова - подстановок вместо части исходной цепочки другой цепочки

К слову $\varphi\alpha\psi$ применяем $\alpha \to \beta$ получаем $\varphi\beta\psi$

- Алгоритм А задается конечным числом правил-подстановок α_i в β_i, реализующих элементарные отображения слов:

  $$A = \{ \alpha_i \to \beta_i \mid i=1,\ldots,k \}$$

- Алгоритм состоит в поиске в текущем слове подслова α_i, и, если оно найдено, то самое левое α_i заменяется на β_i Вход – любое слово в алфавите Σ.
- Например, aababba применением правила aba $\Rightarrow$bccb заменится на abccbbba
- Поиск возможной подстановки происходит, начиная с первого правила в порядке номеров правил
- Алгоритм завершается, когда к текущему слову невозможно применить ни одного из правил, или применяется одно из "заключительных" правил

Пример нормального алгоритма Маркова

Задача: построить алгоритм, вычисляющий K+1 для любого троичного числа K

Пример: при K= 2102: 2102+1=2110 $\Sigma = \{0, 1, 2, \#, +\}$

Схема алгоритма: (= совокупность правил подстановки)

1. #0 →0#
2. #1 →1#
3. #2 →2#
4. # →+
5. 0+ →1
6. 1+ →2
7. 2+ →+0
8. + →1

Пояснения:

Правила 1, 2, 3	– перегоняем маркер # вправо
Правило 4	– вносим спец. символ '+'
Правила 5, 6	– увеличение без переноса
Правило 7	– перенос сложения в следующий разряд
Правило 8	– если перенос оказался перед числом

Пример работы алгоритма при K= 2102: аргумент '#2102'

$\#2102 =_3\Rightarrow 2\#102 =_2\Rightarrow 2\#102 =_1\Rightarrow 210\#2 =_3\Rightarrow 2102\# \Rightarrow$

$=_4\Rightarrow 2102+ =_3\Rightarrow 210+0 =_5\Rightarrow 2110$ (ничего уже применить не можем!)

Пример 2 нормального алгоритма Маркова

Алгоритм выбрасывает комментарии из текста (два слэша и до конца строки).
Перед текстом стоит спец символ #. Другой спец символ - $.
В тексте: ↓ – перевод строки, eot- конец текста, s – любой, кроме # и $)

Алгоритм:

1. #// →$
2. $↓ →↓#
3. #eot →eot
4. $eot →eot
5. #s →s#
6. $s →$

$\Sigma = \{\#, \$\} \cup$ все символы языка

Правило 1 – нашли начало комментария
Правило 2 – после комментария нашли перевод строки
Правила 3, 4 – нашли конец текста
Правило 5 – очередной символ – символ текста
Правило 6 – очередной символ – символ комментария

Пример работы алгоритма при входе `'# begin // начало↓ xy/3=4; eot'`

begin // начало↓ xy/3=4; eot =5*⇒
begin #// начало↓ xy/3=4; eot = 1 ⇒
begin $ начало↓ xy/3=4; eot = 6*⇒
begin $↓ xy/3=4; eot = 2 ⇒
begin ↓ #xy/3=4; eot = 5*⇒
begin ↓ xy/3=4; #eot = 3 ⇒
begin ↓ xy/3=4;eot

Тезис : Алгоритм – это нормальный алгоритм Маркова

- Если для решения некоторой задачи разработан нормальный алгоритм, то эта задача алгоритмически разрешима
- Очевидна справедливость этого утверждения. Действительно, нормальный алгоритм Маркова удовлетворяет неформальному определению алгоритма: он состоит из конечного числа простых выполняемых шагов
- Факт: Все известные до сих пор алгоритмы могут быть представлены нормальными алгоритмами Маркова

Тезис Маркова: если для решения задачи **Т** существует алгоритм, то можно построить эквивалентный ему нормальный алгоритм

`Алгоритм существует' ⇒ `можно построить нормальный алгоритм'

Контрапозиция тезиса Маркова: если для решения задачи Т НЕЛЬЗЯ построить нормальный алгоритм, то никакого алгоритма решения задачи **Т** не существует

`Нельзя построить нормальный алгоритм' ⇒ `Алгоритма не существует'

Другие примеры ассоциативных исчислений

- ***Исчисления Цейтина.***
- ***Проблема выводимости в аксиоматической теории*** – аналогия с ассоциативным исчислением – представляет процесс формальных преобразований с использованием подстановок в аксиомах. Для любых слов R и S требуется доказать, что существует цепочка подстановок (алгоритм), преобразующая R → S.
  - *Теорема Черча доказывает неразрешимость выводимости в виде конечного алгоритма.*
- ***Синтаксический разбор предложений алгоритмического языка***. Синтаксис определяется множеством подстановок, применимых к цепочке символов алфавита языка, расширенного промежуточными – терминальными символами. Если цепочка символов прочитана и распознана применением подстановок, то цепочка принадлежит языку. Схему применения подстановок, выполненную в виде читающего и записывающего автомата, которая формирует символическую строку команд, называют **компилятором**.

Алфавит ∑={a, b, c, d, e}

ac □ ca

ad □ da

bc □ cb

bd □ db

abac □ abace

eca □ ae

edb □ be

abcde □ acbde □ cabde □ cadbe □

Представление алгоритмов автоматами

Конечные автоматы, машины Поста и машины Тьюринга



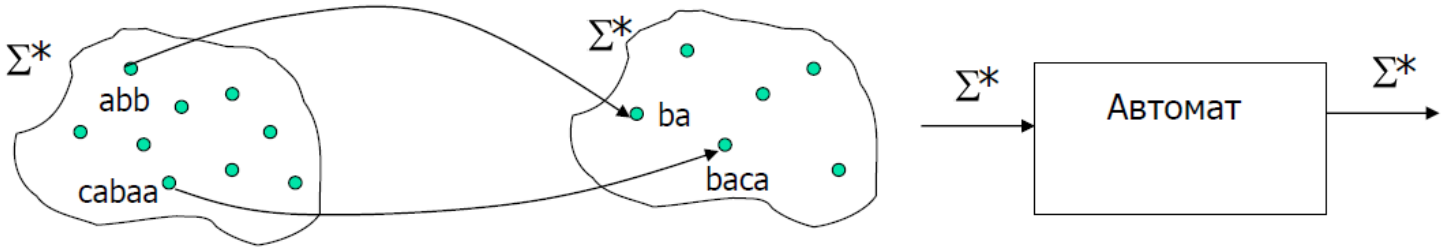
“Механическое” представление алгоритма

Эмилем Постом и Аланом Тьюрингом независимо друг от друга в 1936-1937гг. было дано следующее формальное понятие алгоритма:

Алгоритмический процесс – это процесс, который может совершить подходящим образом устроенная ”машина”

- Постом и Тьюрингом были описаны абстрактные устройства, которые получили название «Машина Поста» и «Машина Тьюринга» (МТ)

Объекты, над которыми выполняется алгоритм



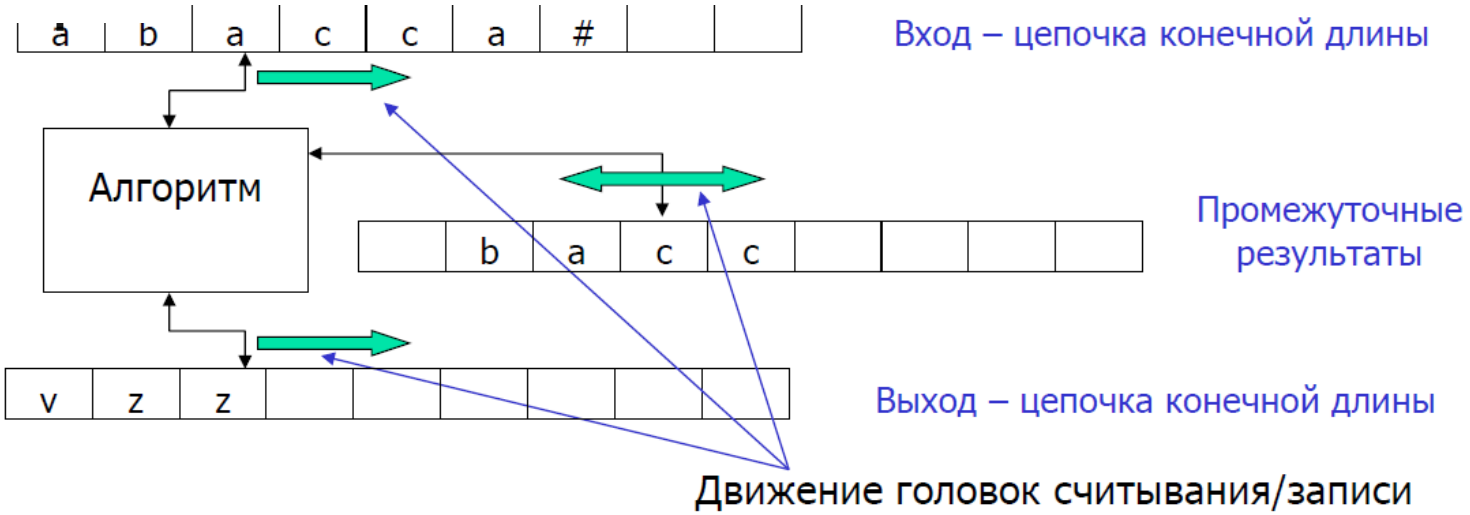
Σ - объединенный конечный словарь входных и выходных символов.

Если Σ={a, b, c}, то Σ*={ε, a, b, c, aa, ab, ..., bcaac, ... }

Конечные автоматы: Частный случай автоматов

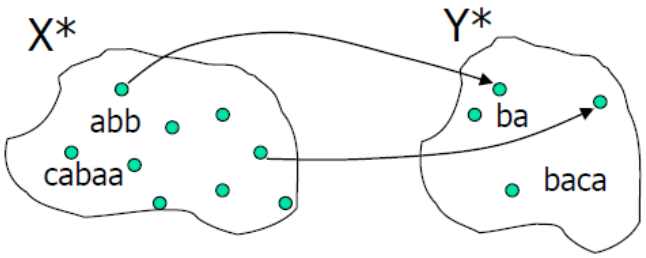
ВОПРОС: Какой класс алгоритмов может быть представлен конечными автоматами (независимо от числа состояний автоматов, их функций переходов и выходов)?

Представление алгоритма как вычислительной процедуры



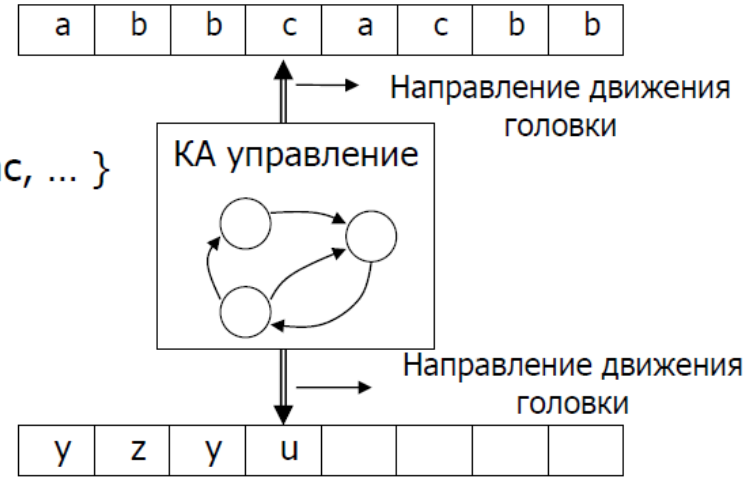
- При выполнении любого алгоритма мы можем считывать входные параметры, пользоваться потенциально неограниченной памятью, запоминая в процессе выполнения алгоритма по мере необходимости нужную информацию, например, на листочке бумаги, и выдавая на выход получающиеся результаты
- Ограничением конечного автомата является конечность числа его состояний, а значит, его памяти

Конечный автомат как алгоритм



X={a, b, c} X*={ε, a, b, c, aa, ab, …, bcaac, … }

- Конечный автомат-преобразователь входных цепочек может представлять алгоритмы



У конечного автомата нет промежуточной памяти: он все запоминает в состояниях

Вместо двух функций, δ и λ, будем использовать одну, выдающую и новое состояние, И выходной сигнал δ: S×X→S×Y

$A=(S, X, Y, s_0, \delta)$

δ: S×X→S×Y

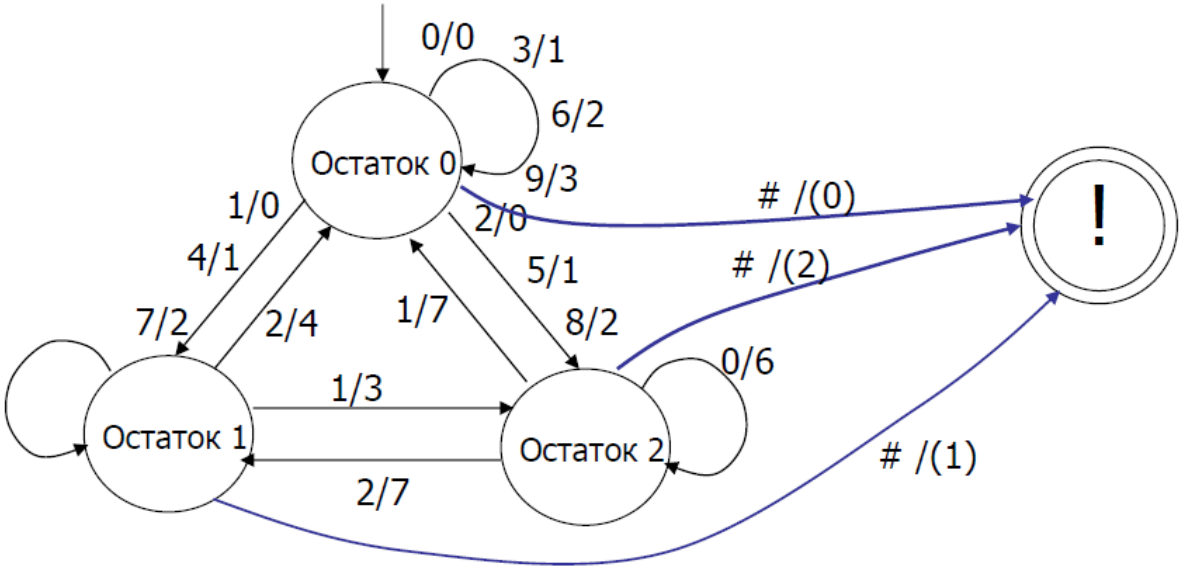
Программа

1. (s,a) →(q, y)
2. (p,b) →(s,z)
3. …

Пример КА как механизма для представления алгоритмов

- Алгоритм деления произвольного десятичного числа на 3
- Вход – десятичное число, заканчивающееся ‘#’
- Выход - <частное>(<остаток>)

Пример работы: 50228# => 16742(2)

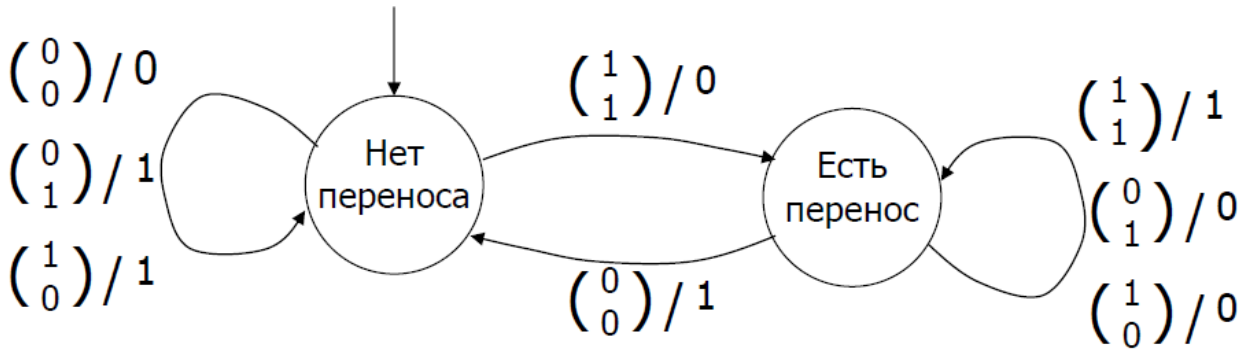


КА: последовательный сумматор

```
  A = 0 0 0 0 1 1 0 1 0 1
+ B = 0 1 0 0 0 1 1 1 1 0
  -----------------------
  C = 0 1 0 1 0 1 0 0 1 1
```

Как построить сумматор, принимающий два числа с младших разрядов, и выдающий их сумму?

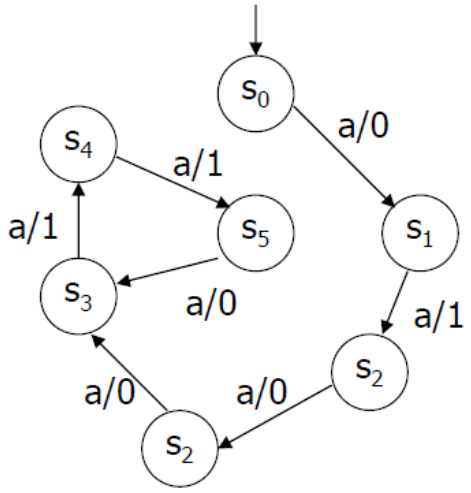
Это должен быть преобразователь с памятью, запоминающий перенос



КА имеет только два состояния, запоминающие перенос 0 и перенос 1

Пример. Генерация периодических последовательностей. Автономный КА

Автономный автомат – конечный автомат, у которого существует единственный входной сигнал (вход синхронизации)



Выход – периодическая последовательность:

0100110110110110...

- Автономным конечным автоматом могут быть сгенерированы любые периодические последовательности
- Период последовательности не может превышать N – числа состояний конечного автомата

КА – (!) Алгоритм. Алгоритм - (?) КА

- **Определение**. *Алгоритм - это определенное на некотором языке конечное предписание (способ, рецепт), задающее дискретную последовательность исполнимых элементарных операций. Процесс выполнения предписания состоит из отдельных шагов, на каждом из которых выполняется одна очередная операция*
- **Утверждение**. Если для решения некоторой задачи разработан конечный автомат-преобразователь, то эта задача алгоритмически разрешима
- *Очевидна справедливость этого утверждения*. Действительно, конечно-автоматный алгоритм удовлетворяет неформальному понятию алгоритма: он состоит из конечного числа простых выполняемых шагов над входными цепочками
- Число различных конечных автоматов бесконечно!
- Число алгоритмов также бесконечно
- Является ли КА средством представления ***любых*** алгоритмов?
- ***ВОПРОС: Можно ли построить (в принципе) конечный автомат для любого алгоритма обработки входных строк?***
- **ОТВЕТ: НЕТ!**

Последовательный мультипликатор

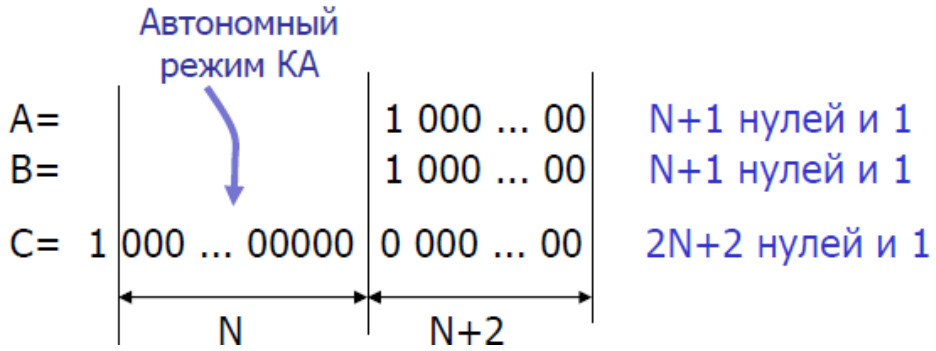
```
  A = 0 0 0 0 1 1 0 1 0 1
× B = 0 1 0 0 0 1 1 1 1 0
-----------------------------
  C = ...
```

Как построить систему умножения, принимающую два двоичных числа с младших разрядов, и выдающий их произведение?

Теорема. Не существует КА, выполняющего алгоритм умножения двух произвольных двоичных чисел

Доказательство. Предположим противное, т.е. что такой автомат существует

Поскольку это КА, то у него конечное число состояний. Пусть это число N. Подадим на его вход $A=B=2^{N+1}$. Тогда C должно быть равно 2^{2N+2}



N+1 нулей и 1

N+1 нулей и 1

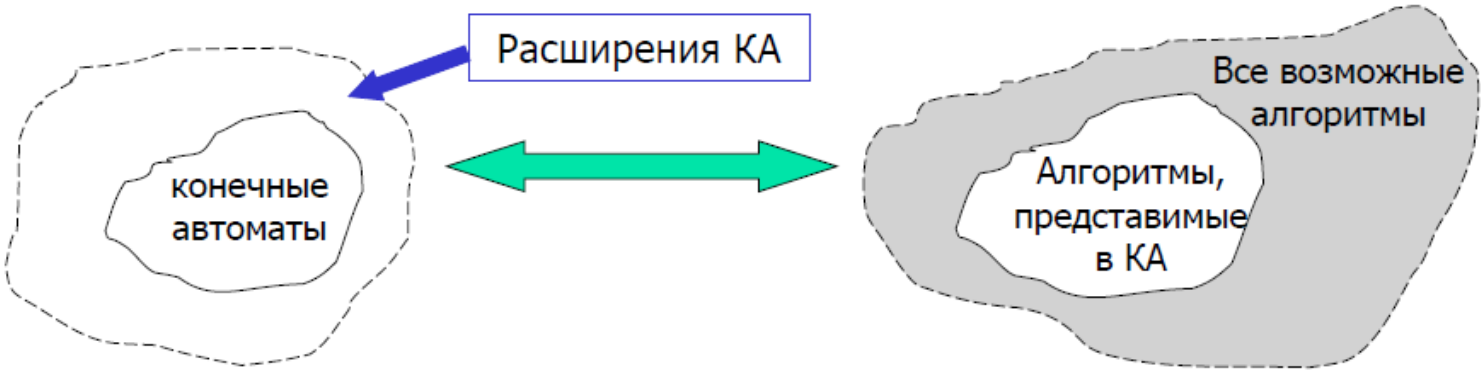
2N+2 нулей и 1

Автомат с N состояниями в автономном режиме не может выдать N нулей, а затем 1

Конечные автоматы не могут выполнить некоторые алгоритмы

КА может выполнить сложение ЛЮБЫХ двоичных чисел,
но не может выполнить умножение двоичных чисел

Вывод: КА как автоматические устройства для выполнения алгоритмов ограничены: существуют такие алгоритмы, которые не могут быть реализованы с помощью КА

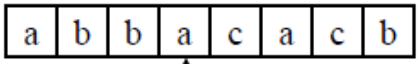


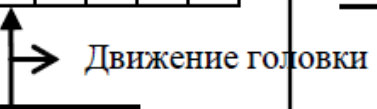
КА не могут выполнить алгоритмы, в которых требуется потенциально неограниченная память

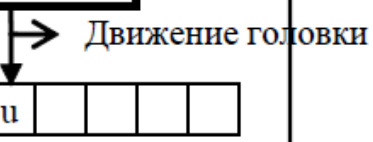
Машины Тьюринга

Конечный автомат

Входная лента







Выходная лента

$A=(S,X,Y,s0,\delta)$

$\delta: S\times X \to S\times Y$

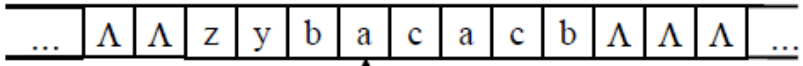
Программа:

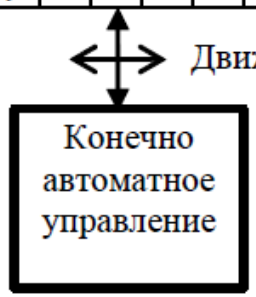
1. $(s,a)\to(p,y)$;
2. $(q,b)\to(s,z)$;
3. ...

Конечное множество четверок

Машина Тьюринга (1936 г.)

Рабочая лента





3 части:

1. Процессор (КА)
2. Головка чтения-записи
3. Бесконечная лента

$MT=(S,X,s0,\delta,\Gamma)$

$\Gamma=\{L,R,H\}$;

$\delta: S\times X \to S\times X\times \Gamma$;

Программа:

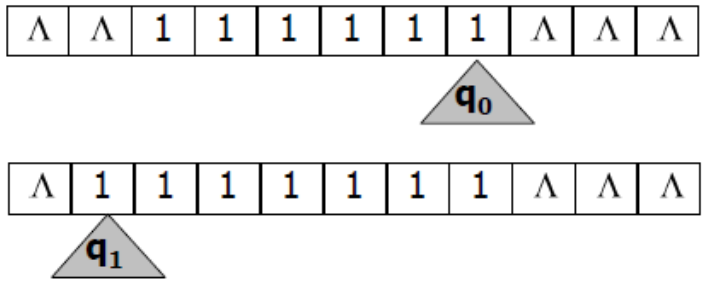
1. $(s,a)\to(p,y,L)$;
2. $(q,b)\to(s,z,R)$;
3. ...

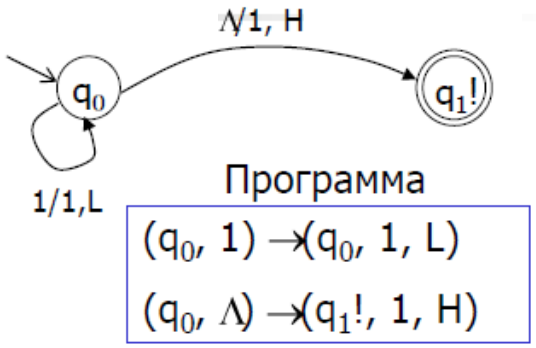
Машина Тьюринга проще КА!

Конечное множество пятерок

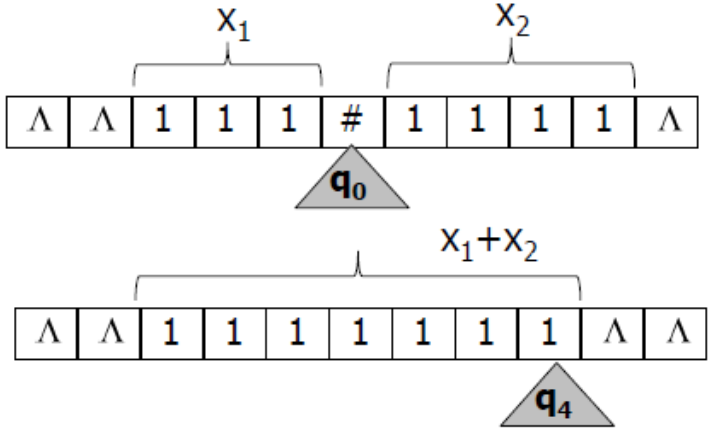
Примеры машин Тьюринга

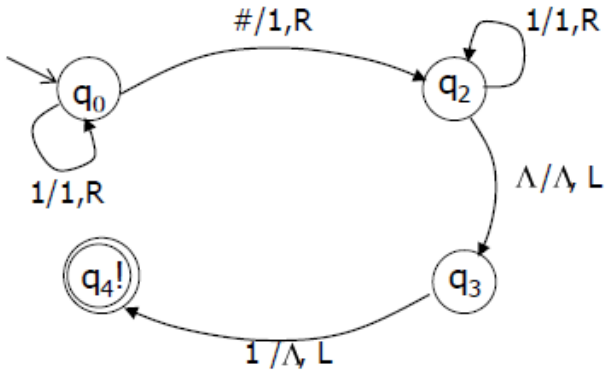
- Функция f(x)=x+1 вычислима



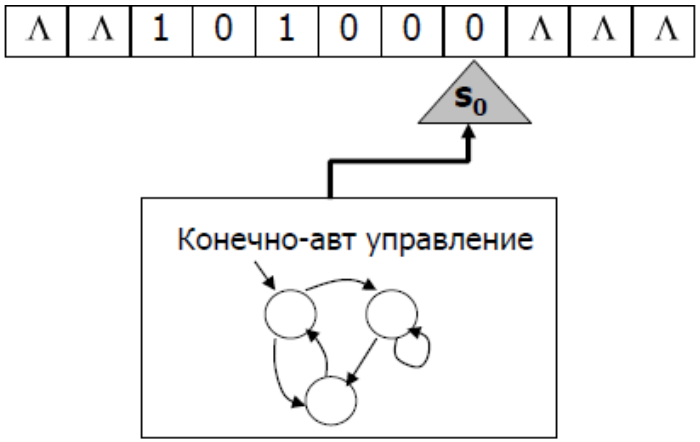


- Функция $f(x_1, x_2)=x_1+x_2$ вычислима

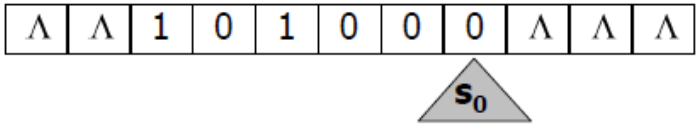




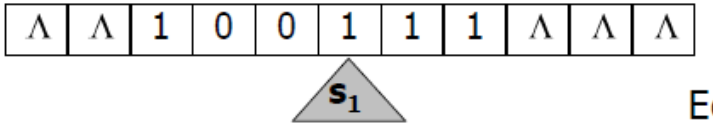
Пример машины Тьюринга: вычитание 1 из двоичного числа



Начальная конфигурациия K_0

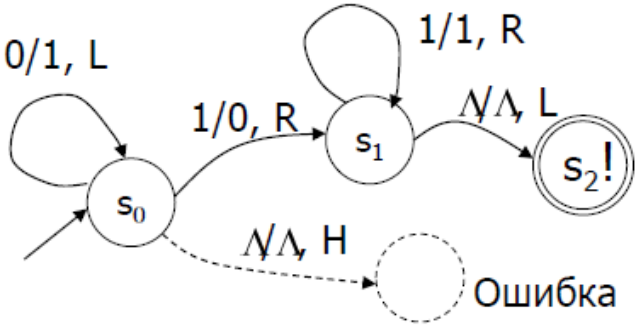


Промежуточная конфигурациия



… …

Граф переходов машины Тьюринга



Программа машины Тьюринга

$(s_0, 0) \rightarrow (s_0, 1, L)$

$(s_0, 1) \rightarrow (s_1, 0, R)$

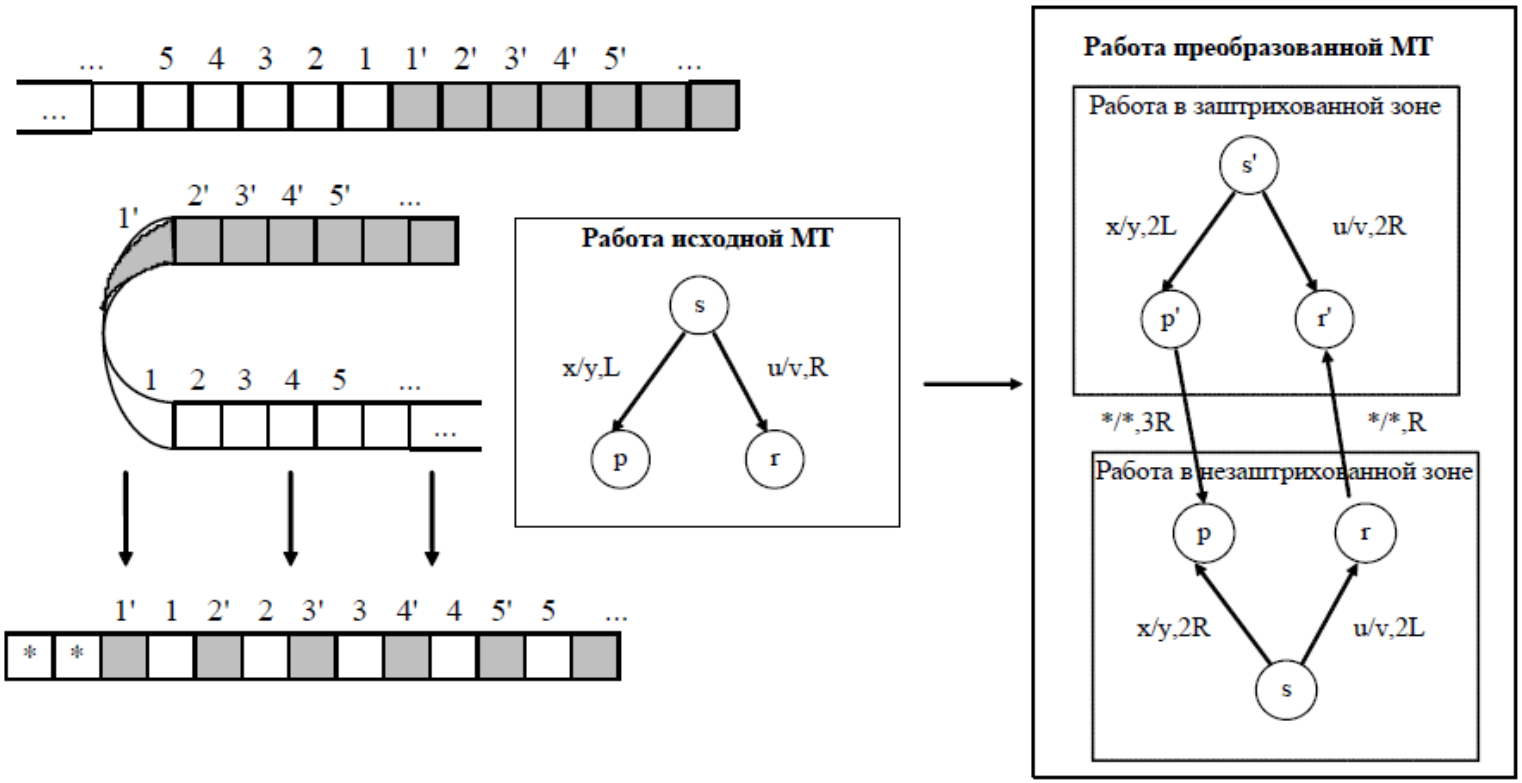
$(s_1, 1) \rightarrow (s1, 1, R)$

$(s_1, \Lambda) \rightarrow (s_1, \Lambda, L)$

Если МТ, начиная работу с конфигурации K_0, не останавливается, то она не применима к K_0

Преобразование машины Тьюринга

- Для любой МТ существует эквивалентная МТ, работающая на полубесконечной ленте

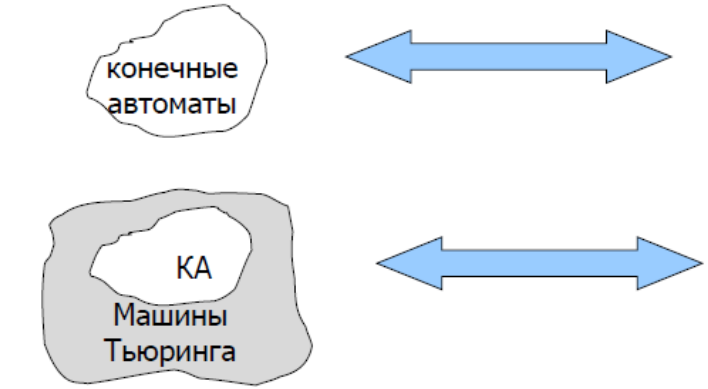


Более сложные вычисления требуют более сложных элементарных базовых правил??? – НЕТ!

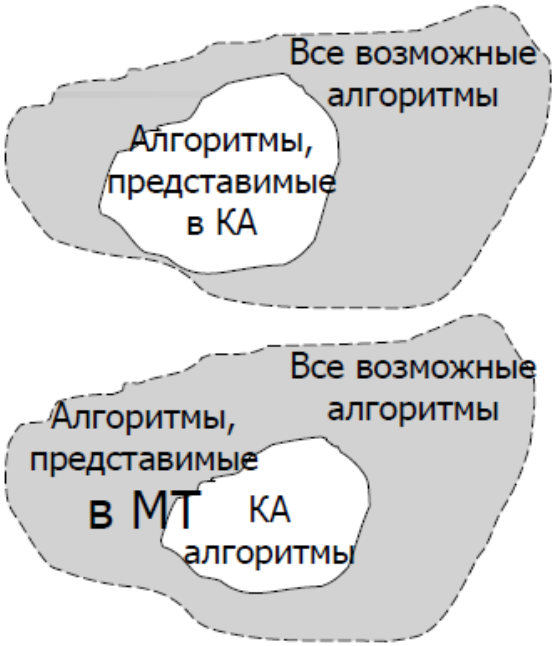


Traditional intuition might suggest that to do more sophisticated computations would always require more sophisticated underlying rules.
But what launched the whole computer revolution is the remarkable fact that universal systems with fixed underlying rules can be built that can in effect perform ANY possible computation.
(Stephen Wolfram. A New Kind of Science)

Тезис Черча-Тьюринга



Тезис Черча-Тьюринга.



Если существует алгоритм решения задачи Т, то его можно реализовать на машине Тьюринга

Контрапозиция этого утверждения.

Если алгоритм решения задачи Т нельзя реализовать на машине Тьюринга, то такого алгоритма НЕ существует

КАК это ДОКАЗАТЬ?? НИКАК!!

“Алгоритм <-> Машина Тьюринга”

- Оказалось, что все известные определения алгоритма эквивалентны

Формулировка тезиса Черча-Тьюринга:

- “*Любое разумное определение алгоритма, которое может быть предложено в будущем, окажется эквивалентным уже известным определениям*”, что означает, по сути, предположение об ***адекватности понятий алгоритма в интуитивном смысле и алгоритма в точном смысле в одном из эквивалентных формализмов: Машина Тьюринга, машина Поста, нормальный алгоритм Маркова, рекурсивные функции, …***
- Это положение используется в качестве *гипотезы*, обоснованной в силу того, что не удалось найти противоречащих ей примеров. Эту гипотезу невозможно доказать **строго**, поскольку понятие алгоритма в интуитивном смысле является неформальным
- Но ее можно будет опровергнуть, если вдруг будет построен алгоритм, который нельзя реализовать на машине Тьюринга

- **Итак ГИПОТЕЗА: “все существующие алгоритмы, и все алгоритмы, которые когда-нибудь могут быть созданы, могут быть реализованы на МТ”**

Универсальный язык записи алгоритмов: Тьюрингол

Команда	Пояснение
сдвиг $\{L, R\}$	Сдвиг головки на 1 позицию на ленте влево или вправо (увеличить или уменьшить адрес следующего считываемого символа)
переход p	Безусловный переход к команде с номером p
если s, p	Условный переход к команде с номером p, если в обозреваемой ячейке находится s
печать s	Печать символа s в обозреваемую ячейку
стоп	Останов

Программа работает с потенциально бесконечным массивом символов

Символы – элементы конечного алфавита (их можно определить, как перечислимое множество)

Следствие тезиса Черча-Тьюринга:

Для того, чтобы автоматическое вычислительное устройство могло реализовать любой мыслимый алгоритм, достаточно, чтобы оно могло:

1. *работать с линейной памятью - аналогом полубесконечной входной ленты машины Тьюринга, и*
2. *выполнять, по крайней мере,* ***пять перечисленных выше операций***

МТ и концепция универсальных вычислений

Все существующие и будущие компьютеры фундаментально эквивалентны, несмотря на то, как они построены

- Все существующие алгоритмы, и все алгоритмы, которые когда-нибудь могут быть созданы, могут быть построены только из нескольких простейших примитивных "кирпичиков!", простейших команд, соответствующих командам МТ, простым их соединением в том или ином порядке

Удивительный, совершенно необычный результат!!

Является ли задачей извлечение квадратного корня, вычисление баллистической траектории ракеты, компьютерная игра, рисование изображений или согласование банковской транзакции – в основе нее лежат простейшие команды манипуляции с символами, и Машина Тьюринга может быть запрограммирована, чтобы выполнять ее (Джеф Хокинз. "Об интеллекте")

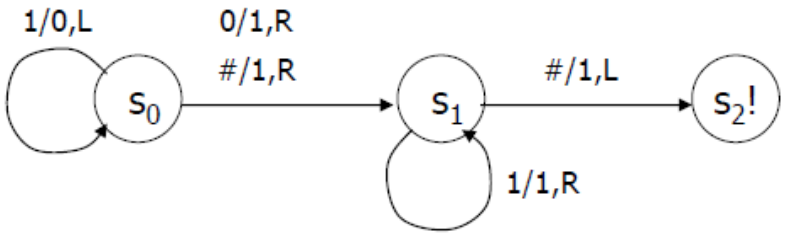
Пример программы на Тьюринголе

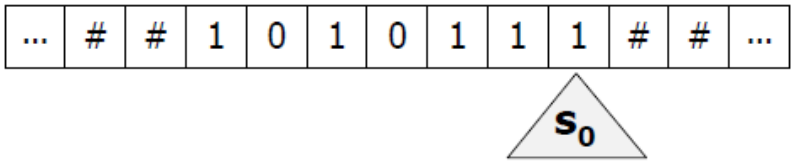
```
00: если      1,  02
01: переход   05
02: печать    0        // встретили 1
03: сдвиг     L
04: переход   00       // на s0
05: если      0, 07
06: переход   10
07: печать    1        // встретили 0
08: сдвиг     R
09: переход   16       // на s1
10: если      #, 12
11: переход   15
12: печать    1        //  встретили #
13: сдвиг     R
14: переход   16       // на s1
15: стоп               // ошибка входа
16: если      1, 18
17: переход   21
18: печать    1        // встретили 1
19: сдвиг     R
....
```

S_0

S_1

Увеличение двоичного числа на 1





Любой ЯВУ включает эти команды. Следовательно, любой, более или менее богатый язык программирования является УНИВЕРСАЛЬНЫМ

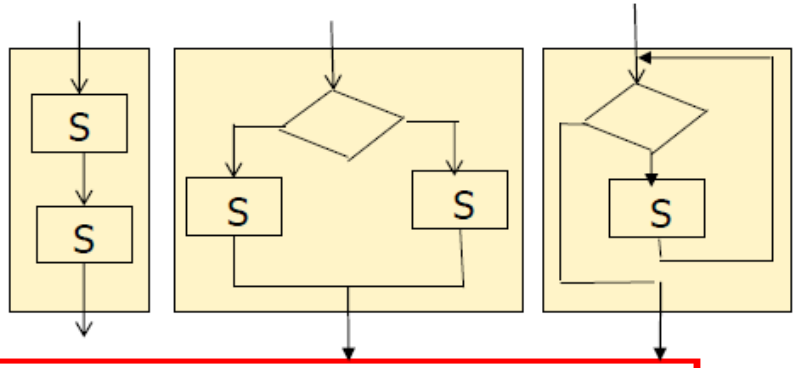
Теорема структурного программирования: можно ли избавиться от “спагетти-кода”

В 60-х годах стало понятно, что программы пишутся НЕ ДЛЯ компьютера, а для ЧЕЛОВЕКА: использование go_to для *исполнения* в программе ничуть не труднее выполнения любой операции. Однако понимание *человеком* программы неумеренное использование go_to сильно усложняет

Дейкстра: ("Go To Statement Considered Harmful", CACM, Vol. 11, No. 3, 1968)

*"За многие годы я утвердился во мнении, что квалификация программистов - функция, обратно зависящая от частоты появления операторов **go to** в их программах. Позже я открыл, почему оператор **go to** производит такой пагубный эффект, и я убежден в том, что оператор **go to** должен быть отменен в языках программирования "высокого уровня"*

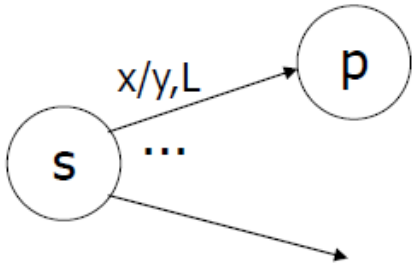
Теорема Бома-Якопини: “Любой алгоритм можно реализовать на языке высокого уровня с использованием только трех управляющих конструкций: последовательность, выбор и цикл



Как ее доказать? Для конкретного языка (Паскаль) – можно, но как доказать эту теорему для ЛЮБОГО языка?!!!

Доказательство теоремы структурного программирования

```
begin
...
state = s0 ;                          // устанавливаем начальное состояние
while ( state != стоп ) do {          // повторяем цикл, пока не придем к останову
    Читать (symbol);
    ...
   if (state == s && symbol == x ) then  // реализация команды МТ  (s,x) →(p,y,L)
        state  = p ;
        Писать (y) ;
        Сдвиг (L)
  else
   if (state == ...    ) then
         ...
         ...
   end if
end while
```



Любую машину Тьюринга (и, следовательно, **любой алгоритм**) можно реализовать на языке высокого уровня с использованием только трех управляющих конструкций: **последовательность, выбор и цикл (без goto)**.

Доказали известную теорему Бома-Якопини

Немного о языках программирования ☺



Доказательство существования алгоритмически неразрешимых проблем

Проблема останова



Существование алгоритмически неразрешимых задач

- **Теорема. Существуют алгоритмически неразрешимые задачи.**
- **Доказательство.** Задачи построения отображения f:NN являются подклассом возможных задач. Таких задач – бесконечное число (континуум).
- Все возможные алгоритмы – это конечные предписания, выраженные в языке с конечным числом символов, и их можно лексикографически упорядочить. И, следовательно, пересчитать.
- Итак, число возможных алгоритмов счетно. Но поскольку число задач несчетно, то существует бесконечное число задач, для которых нельзя построить решающих их алгоритмов.

Иными словами, существуют алгоритмически неразрешимые задачи, т.е. такие задачи, для которых нельзя построить алгоритма решения



Проблема существования совершенных чисел

- Совершенные числа
  - Совершенное число – Это число, которое равно сумме своих делителей, например:
    6 = 1+2+3;
    28 = 1+2+4+7+14
- Проблема существования совершенных чисел:
  Существует ли нечетное совершенное число? Никто не знает ответ!

Построим программу из четырех строк:

```
for( int x=1; true;  x+=2 )
    begin
        int s = 0;
        for( int i =0;  i<=x/2;  i++ )  if (x %i == 0) s += i;
        if s == x  break
    end
```

Если программа остановится, то такое число существует

Остановится ли этот алгоритм? Никто не знает!

Можно ли по записи алгоритма определить, остановится ли он на конкретных исходных данных??

Проблема Гольдбаха

- Теорема Гольдбаха: Любое четное число, большее 2, может быть представлено, как сумма двух простых чисел

```
bool  Goldbach( )
begin
   for  (int n=4;  true;  n:=n+2)
   begin
      L := True;
      for( int m=2; m<n/2; m++)
      if(Простое(m)&&Простое(n-m)) then L:=False;
      if L then stop // не нашлось пары простых для n
    end
end
```

Останавливается ли этот алгоритм? Никто не знает!

Если по виду алгоритма и его исходных данных D можно определить, останавливается ли этот алгоритм на исходных данных D, или нет, то можно доказать или опровергнуть проблему Гольдбаха не проводя вычислений!



Неразрешимость проблемы останова

Теорема. Не существует алгоритма, который по описанию произвольного алгоритма и его исходных данных определяет, останавливается ли этот алгоритм на этих данных, или работает бесконечно

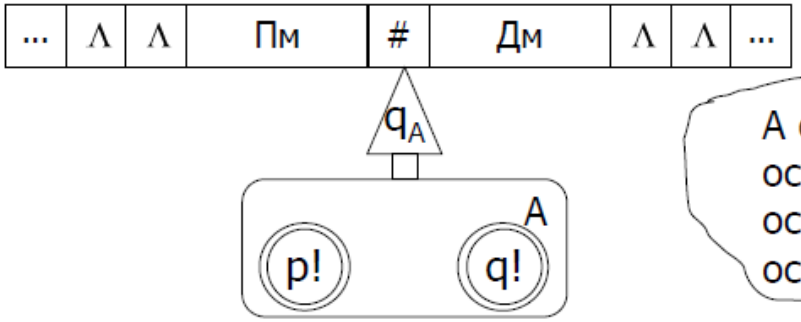
Доказательство основано на тезисе Черча-Тьюринга.
Докажем, что не существует машины Тьюринга, которая по описанию произвольной МТ и ее входных данных определяет, останавливается ли она, или работает бесконечно



Утверждение: проблема останова Машины Тьюринга неразрешима

По предположению, существует МТ А, которая на Пм и ее входных данных Пм останавливается в p!, если М на этих входных данных не останавливается

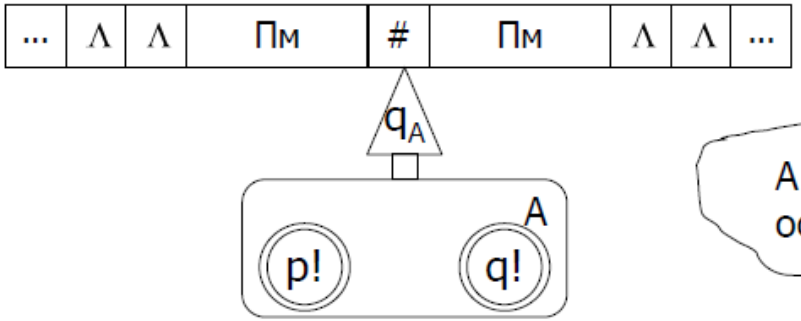
Предположим противное! Такая машина Тьюринга А существует для любой М



Пм – программа машины М

А останавливается в p!, если М не останавливается на данных Дм, и останавливается в q!, если М останавливается на Дм

Частным случаем Дм является описание самой машины Тьюринга М, т.е. Пм



А останавливается в p!, если М не останавливается на Пм

Ясно, что если существует А , то существует и МТ В, которая копирует описание Пм, дублируя его, а потом уже работает, как А

Примеры алгоритмически неразрешимых проблем

- Диофантовы уравнения $a^n+b^n=c^n$ в области натуральных чисел при n, больших 2. Такого алгоритма не существует
- Десятая проблема Гильберта: существует ли алгоритм, который определяет, имеет ли целочисленные корни уравнение P=0, где P - многочлен n-ой степени с целыми коэффициентами?
  Такого алгоритма не существует
- Проблема эквивалентности алгоритмов: По двум произвольным заданным алгоритмам (например, двум программам С) определить, будут ли они выдавать одинаковые выходные результаты на любых исходных данных. Такого алгоритма не существует
- Неразрешимость элементарной арифметики. Любое утверждение о натуральных числах может быть выражено предикатом первого порядка. Не существует алгоритма, который может проверить, истинность или ложность таких предикатов



Примеры алгоритмически неразрешимых проблем

- Диофантовы уравнения - это система многочленов с целыми коэффициентами, и решение этой системы ищется в целых числах. Невозможность построения алгоритма в общем случае вовсе не означает, что нельзя решить какую-то конкретную задачу
- Относительно диофантовых уравнений известно, что невозможно создать алгоритм, который бы мог за конечное количество шагов определить, имеет ли данная система диофантовых уравнений решение или нет



Следствия существования алгоритмически неразрешимых проблем

- Алгоритм – способ решения класса задач. Для некоторых классов задач алгоритма их решения не существует (алгоритмически неразрешимые проблемы)
- Существование алгоритмически неразрешимых проблем является удивительным и очень мощным результатом как с философской, так и с практической точки зрения. Нельзя ставить проблемы и строить алгоритмы, не анализируя детально поставленную проблему: можно по незнанию искать алгоритм, которого нет в природе
- То, что некоторые проблемы в их ОБЩЕЙ постановке неразрешимы, вовсе не значит, что для конкретной постановки проблемы, или даже для целых подклассов КОНКРЕТНЫХ постановок этой проблемы их нельзя решить
- Обычно при разработке алгоритма следует учитывать ОСОБЕННОСТИ конкретного подкласса проблем
- *Например, останавливается ли алгоритм НОД? Можно доказать, что этот алгоритм останавливается для любой пары натуральных чисел*
- Для некоторых хорошо поставленных математических проблем НЕИЗВЕСТНО, являются они алгоритмически разрешимыми, либо нет

Следствия из тезиса Черча-Тьюринга

Чем полезна теория со строгим определением понятия алгоритма?

- Она позволяет решить проблемы:
  - разработки универсального языка для записи любых возможных алгоритмов, даже если они еще не придуманы
  - проблему построения универсальных автоматических устройств, на которых можно реализовать любые возможные алгоритмы
  - проблему существования алгоритмов: есть ли какие-нибудь принципиальные ограничения для построения алгоритмов?
  - проблему оценки сложности алгоритмов и сравнения алгоритмов между собой по их сложности

- Решение проблем:
  - любой язык, на котором можно описать работу МТ, будет универсальным – на нем В ПРИНЦИПЕ можно записать любой алгоритм, который будет придуман
  - универсальные автоматические устройства - это любые процессоры, на которых можно реализовать команды МТ. Таких команд всего 5!!
  - если покажем, что не может существовать МТ, решающая некоторую проблему, то алгоритма решения этой проблемы НЕ СУЩЕСТВУЕТ. Это – контрапозиция тезиса Черча-Тьюринга. Найдено много неразрешимых проблем
  - проблема оценки сложности алгоритмов решается так: оценивается сложность (необходимое время и память) для работы алгоритмов, реализованных на МТ

Заключение

- *Понятие алгоритма является базовым понятием* современной науки. Это понятие невозможно определить единственным образом, но оно существует объективно
- *Многие определения понятия алгоритма оказались эквивалентными*: лямбда исчисление, рекурсивные функции, машина Поста, машина Тьюринга, алгоритмы Маркова, ...
- **Тезис Черча-Тьюринга**: Если задача имеет алгоритмическое решение, то это решение может быть представлено машиной Тьюринга (и другими формализмами)
- Существуют строго определенные проблемы, для которых не может *существовать разрешающих их общих алгоритмов*. Неразрешимых проблем много. Примеры: проблема останова, решение диофантовых уравнений. Невычислимость какой-либо задачи означает невозможность решить ее за конечное количество шагов.
- Неразрешимость проблемы в общей постановке не означает того, что в более частной постановке проблема НЕ может быть решена



Терминология

- **Теория вычислимости (computational theory)** – теория, изучающая вопросы существования алгоритмов (теория алгоритмов)
- **Теория вычислений (computing theory)** – вопросы численных методов приближенного решения дифференциальных уравнений, интегро-дифференциальных уравнений и т.п.
- **Информатика (computer science)** – изучение конкретных алгоритмов, структур данных, реализации алгоритмов на компьютере, в языках программирования, ...
- **Теория сложности вычислений (complexity theory)** - анализ сложности алгоритмов - вопросы того, за какое число шагов (либо с каким объемом памяти) можно вычислить значение вычислимой функции



Результаты

Основной результат *теории вычислимости*:

- Не для всех четко сформулированных проблем могут быть в принципе построены алгоритмы, решающие эти проблемы. Существуют АЛГОРИТМИЧЕСКИ НЕРАЗРЕШИМЫЕ проблемы Это удивительный, очень нетривиальный и неожиданный результат

Основной результат *теории сложности вычислений*:

- Для различных классов проблем могут быть построены оценки сложности алгоритмов их решения, которые определяют рост времени и/или памяти при росте параметра решаемой проблемы. Эти оценки НЕ ЗАВИСЯТ от природы и сложности устройства, выполняющего алгоритм. Например, от марки компьютера
- В частности, для некоторых классов проблем доказана их высокая сложность, что говорит о том, что даже при очень небольших размерах проблемы ее практически невозможно решить независимо от быстродействия используемого компьютера

Персоналии

Готфрид Вильгельм Лейбниц (1646-1716), философ, математик, физик, 300 лет назад поставил задачу нахождения универсального алгоритма решения всех математических проблем, если они выражены в формальном языке логики предикатов первого порядка. Эта задача стала важнейшей философской и математической проблемой. В 1936 г. невозможность существования алгоритма выводимости в классическом исчислении предикатов была независимо доказана Алонзо Черчем и Аланом Тьюрингом

Алонзо Чёрч (1903-1995) построил формальную систему, названную им лямбда-исчислением, и определил в ней понятие алгоритмической вычислимости

Алан Тьюринг (1912-1954) предложил абстрактное вычислительное устройство (задолго до изобретения компьютера), и понятие вычислимой функции определил через эту машину

- Тьюринг доказал эквивалентность этих двух формальных моделей (лямбда-исчисления и Машины Тьюринга): обе они определяют один и тот же класс вычислимых функций
- Из идеи машины Тьюринга выросли современные компьютеры и императивные языки программирования. Из идеи лямбда-исчисления выросло современное функциональное программирование - языки ML (Meta Language), Miranda, Haskell ...

Марков, Андрей Андреевич-мл. (1903-1979), разработал теорию подстановок (нормальный алгоритм), формальную процедуру преобразования слов в конечном алфавите как возможную универсальную форму представления алгоритмов

СПАСИБО ЗА ВНИМАНИЕ